SOL ORGANIZES LOGIC / RESEARCH MEMORANDUM

SOL ESSENCE

Interface precedents, Textual integration, and the architecture of an investigative encyclopedia

Prepared for: SOL maintainer and Claude Code (documentation revision only)
Date: Sunday, 18 October 2026
Repository examined: dialectmaterial/SOL, default branch and canonical docs
Scope: research and design decisions; no application code changes

Central recommendation

Keep Neovim as the persistent BEING reader. Launch a separate Textual application only when ESSENCE is requested; let Neovim own the outer split/fullscreen arrangement. Treat the interface as a durable framework for an eventual history-of-models encyclopedia and experiment laboratory, not merely as a tokenizer menu.

Evidence basis: official documentation and project repositories, SOL canonical documentation and packaging metadata, and the design decisions established in the accompanying conversation. External references are listed with stable URLs at the end.

EXECUTIVE POSITION

1. What the project is deciding

SOL ESSENCE is to become an **interactive investigative encyclopedia**: a place to study the history, architecture, inadequacies, and trade-offs of computational methods, then put those methods to work on philosophical corpora. Its long-term purpose is explanatory and experimental. The immediate development goal is much narrower: establish a reversible interface connecting reading in BEING to investigation in ESSENCE.

The proposed stack is a hybrid. **Neovim** remains the persistent, comfortable reader; **Textual** provides ESSENCE's interactive application surface; **SOL CORE** provides a structured, provenance-aware corpus when present. ESSENCE must also run without CORE, accepting plain text and file inputs. This distinction lets the project mature its architecture without prematurely creating a model framework or an operating system.

Six decisions to carry into the documentation

- **Interface before analytical breadth.** First prove launch, split, fullscreen, navigation, state preservation, close, and return; then add word tokenization as the smallest real operation.
- **Two ESSENCE presentations, one investigation state.** Fullscreen and side-by-side with BEING are views of the same active Textual session, not separate apps.
- **On-demand lifecycle.** Textual is absent while the reader is idle. Entering ESSENCE starts its process; closing ESSENCE saves state and terminates the process. Resizing between ESSENCE layouts does not restart it.
- **Separation of responsibilities.** BEING reads; ESSENCE studies methods and orchestrates experiments; CORE owns canonical corpora, identity, and provenance.
- **Model history is not the model picker.** The encyclopedia describes conceptual lineages and evidence; a task/capability registry supplies runnable implementations.
- **Documentation is the immediate deliverable.** Claude Code should reconcile these decisions with the code, update the four canonical documents, preserve historical evidence, and defer runtime implementation.

What this report does not claim

It does not assert that Textual integration, a general model registry, autonomous package distributions, comparative multi-author corpora, or an encyclopedia already exist in SOL. It does not certify that a specific Textual-in-Neovim launch arrangement has been manually tested on the current machine. Those are design choices and acceptance targets.

REPOSITORY EVIDENCE

2. Current SOL baseline and architectural tension

The repository review focused on **pyproject.toml**, README, CLAUDE.md, the four authoritative documents under docs/, and the legacy inventory. The present package is **sol-organizes-logic**, not three published standalone Python distributions. It declares Python 3.10+, includes prompt-toolkit, markdown-it-py and OpenDataLoader PDF in base dependencies, and exposes CLI commands through one packaging unit. [R1-R6]

Current capability / rule	Verified in docs	Implication for this brief
Neovim-backed centered reader	BEING: bounded 64-paragraph sections, source IDs, persistent workspace and current cursor; several Lua modules. [R3]	Do not replace the reader or overwrite established keymaps.
ESSENCE lexical / sentence experiments	ESSENCE: prose export, term study, sentence study; **S** already opens a sentence-length study. [R4]	Treat the new Textual application as proposed; audit shortcut collisions.
CORE corpus/reference contract	CORE: audited Cambridge / di Giovanni edition, UUIDs, fingerprints, verified snapshots. [R5]	Preserve existing verification; plan broader collections as future scope.
Documentation authority	Four current docs; legacy archive explicitly superseded; CLAUDE.md operational only. [R2,R6]	Revise current docs, never silently promote archived plans.

Important tensions Claude Code must resolve, not conceal

- **Book scope:** current CORE expressly limits active development to one audited *Science of Logic* edition. Longer-term cross-work and author-bibliography comparisons should be stated as future requirements, not added as a shipping catalog.
- **Modularity:** the documentation favors distinct responsibilities, yet package installation remains monolithic and ESSENCE presently imports CORE. Specify a credible path toward standalone BEING/ESSENCE with optional CORE integration.
- **Existing reader interaction:** the reader already binds **S** to the sentence study. The new ESSENCE entry key must be designed around existing mappings, with no unilateral remapping.
- **Metadata/claims:** passage identity and reproducibility are core strengths. Any UI/experiment integration must preserve verified source references and distinguish source evidence from model output.

Review limitation: repository source and canonical Markdown were read remotely; no local running Neovim/Textual session or private corpus was executed. The code may be ahead of documentation, and in-progress branches require fresh reconciliation.

COMPARATIVE RESEARCH

3. What to borrow from other terminal interfaces

No single precedent provides the entire SOL experience. The right method is to borrow specific *interaction invariants* rather than visual imitation or wholesale framework migration. The examples below are grounded in project READMEs, developer guides and, where available, implementation documentation.

Program / stack	Verified design strength	SOL lesson
Herdr — Rust terminal multiplexer	Workspaces, tabs, independent panes, detachable sessions, agent state sidebar, keyboard/mouse manipulation. [U1]	Borrow reversible spatial arrangement and visible focus, **not** an inner terminal multiplexer.
tmux — C multiplexer	`resize-pane -Z` toggles focused pane between zoomed and its former layout. [U2]	Two ESSENCE presentations should be reversibly switchable with no state loss.

Program / stack	Verified design strength	SOL lesson
Lazygit — Go/gocui	Contexts tied to views; controllers bind actions; helpers and presentation separate logic. [U3]	Context-sensitive keybindings, navigable results, visible action vocabulary.
K9s — Go TUI	Resource-oriented views, `:` commands, `/` filters and `Esc` view history. [U4]	Search/filter/history should be predictable across encyclopedia and experiment views.
Yazi — Rust	Multi-column browsing, selection/preview, Vim-like motions; async processing and cancelable work. [U5]	Tree-to-detail navigation and nonblocking long operations.

Herdr versus Lazygit: two distinct design grammars

Herdr is spatial: users compose workspaces from persistent independent terminal processes. **Lazygit is contextual:** selecting an object determines relevant commands, related views, and the next action. SOL needs both concepts, but not both implementations. Neovim already supplies outer window geometry; Textual can supply focused widgets, contexts, forms and results. [U1,U3,N1]

Herdr's server-side process persistence is **not** the model to copy literally: the current SOL requirement is that Textual should terminate when ESSENCE closes. It is the *reversible layout* and unambiguous focus model that matter, not immortal pane processes.

Precedent evaluation matrix

Criterion	Herdr / tmux	Lazygit / K9s	Suggested ESSENCE
Changing geometry	Core concept	Usually fixed views	Neovim split/fullscreen toggle
Context-sensitive actions	Secondary	Core concept	Focused Textual widget + command palette
Persistent state	Live process / session	Context and domain state	Serializable investigation state
Independent methods	Terminal processes	Commands on domain objects	Adapters/external model runners

TEXTUAL IMPLEMENTATIONS

4. Real Textual applications worth studying

These examples demonstrate not only that Textual can render layouts, but how production applications divide selection, editing, monitoring, data tables and command access. They vary in maturity and purpose; none should be copied wholesale.

Harlequin — strongest architecture reference

Harlequin is a Python/Textual SQL IDE with a data catalog, editor, query execution and results views. It supports installable database adapters. Its newer companion command **hsql** can run comparable operations without opening the TUI and shares adapter/configuration infrastructure. For ESSENCE, the critical pattern is **one capability with both interactive and headless access**. [A1]

Posting 2 — parameter-rich interaction; Posting 3 — important counterexample

Posting 2 used Textual for API request editing, environment variables and contextual navigation. The project's current Posting 3 beta is a separately evolved **Go/Terma** application; its README explicitly distinguishes the frameworks. Cite **Posting 2** when using Textual as the precedent. The rewrite cautions against assuming that a TUI framework choice is permanent. [A2]

Frogmouth — document and reference navigation

This Textual Markdown browser provides local/remote document viewing, table of contents, bookmarks and a browser-like navigation stack. It is the clearest example for ESSENCE's **historical encyclopedia**, whose entries need bibliographic links, navigation history, and formatted text. It is not a reason to replace the existing BEING reader. [A3]

Toolong — large, inspectable evidence

Toolong supports opening, searching, following and merging log files, including JSONL. Its fast large-file access and tabbed source handling are useful precedents for **experiment logs and result inspection**, not for heavy numerical visualization in the terminal. [A4]

Dolphie — status and temporal observation

Dolphie presents real-time database-monitoring information through Textual and also supports a daemon mode that omits the TUI. The transferable idea is status visibility and a hard boundary between collection/computation and presentation. [A5]

Toad — unified agent workspace

Toad uses contextual key help and a unified terminal interface for coding agents, with command-line entry points and documented future session features. Its footer and help conventions are relevant to an ESSENCE mode with many possible actions. Features listed as roadmap should **not** be treated as implemented. [A6]

Additional ecosystem patterns

The Textualize ecosystem includes **Trogon** (forms for command-line programs) and other Textual examples. These reinforce a restrained approach: leverage existing controls, keyboard help and commands instead of hand-building controls from terminal drawing primitives. [A7,T1]

DESIGN EXTRACTION

5. Translating precedents into Textual components

Textual is a Python application framework built on Rich. It provides declarative widget composition, CSS-like terminal layout, focus/input, reactive events, screens, built-in components, workers and test tooling. It does **not** integrate Neovim automatically. [T1-T4]

Desired SOL interaction	Textual primitive to evaluate	Rationale / guardrail
Browse model lineages	Tree, ListView, MarkdownViewer, Tabs	Keep historical claims source-linked; navigation distinct from runnable registry.
Configure experiment	Select, Input, Switch, OptionList, form validation	Show task-supported parameters; mark unavailable runtimes.
View lexical or model output	DataTable, RichLog, Markdown, ProgressBar	Source-linked rows, explicit denominators and warnings.

Desired SOL interaction	Textual primitive to evaluate	Rationale / guardrail
Navigate many actions	Actions, key bindings, command palette, footer	Help changes with focus; do not override reader keys blindly.
Switch within ESSENCE	Screen, ContentSwitcher, Horizontal/Vertical/Grid	Independent of Neovim outer split/fullscreen geometry.
Execute expensive job	Worker + external process / job runner	UI responsive; heavy CPU/GPU execution outside UI event loop.
Regression-test workflow	Textual Pilot; snapshot tests	Automate focus, key sequences, narrow terminals and refusal states.

Technical boundary that should remain explicit

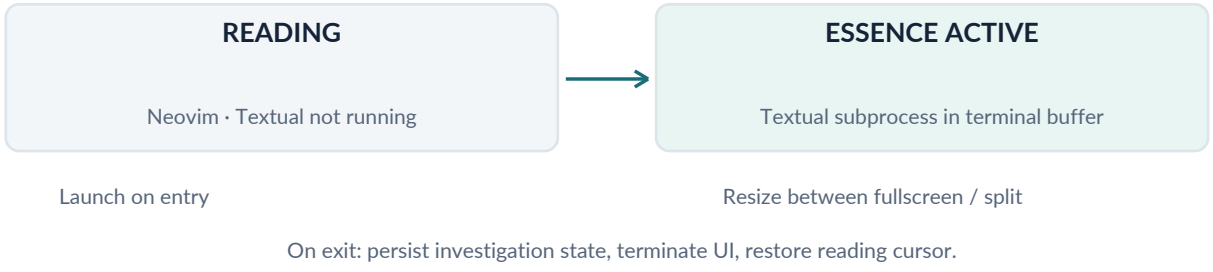
Outer layout: Neovim decides whether its terminal buffer occupies the screen or a split beside BEING. **Inner layout:** Textual decides how panels, forms, encyclopedia pages and results fit within the space assigned to that terminal buffer. Thus “fullscreen ESSENCE” is not necessarily a different Textual Screen; it may be the same application resized to a larger Neovim window. [N1,T1]

Textual Screen switching remains useful *inside* ESSENCE, for example encyclopedia entry versus experiment setup versus results. These are logical navigation screens, not the two outer SOL presentations. Conflating the two would produce brittle resize behavior. [T1,T4]

TARGET INTERACTION

6. One research session, three visible states

The two requested ESSENCE presentations complement the ordinary reading state. There are therefore **three visible states overall** but only **two ESSENCE presentations**. The user may switch ESSENCE fullscreen and side-by-side without restarting its process; the application exists only during an active ESSENCE interval.



Conceptual process lifecycle. Layout transitions inside ESSENCE preserve one active process and one investigation.

State A — BEING alone

Neovim displays the centered prose, source warnings, annotations and familiar motions. Textual is not running. A user can select a passage or ask a question. Entry into ESSENCE carries a source reference, an optional selected span, and the current reading position.

State B — BEING + ESSENCE side-by-side

Neovim creates an ESSENCE terminal buffer in a split to the left or right; the reader remains visible. Textual renders appropriately for its assigned dimensions. Focus can move predictably between windows. The default should be one ESSENCE panel, not two algorithm panes on either side of text.

State C — ESSENCE fullscreen

Neovim expands the same ESSENCE terminal buffer to occupy its available viewport. Textual's inner layout can show a browser, parameters, results, or two comparable result panes. Returning to the split restores the previous arrangement, focus and current investigation.

Representative scenario: target vs reference corpus

- While reading a paragraph, activate ESSENCE and choose **Word frequencies**. The initial target is the current passage, but it is editable.
- Select a reference: section, chapter, entire work, or (future) another author collection. ESSENCE asks CORE to resolve its references when CORE is installed; otherwise it loads ordinary input files.
- Execute the same declared tokenizer on both scopes; show absolute counts and normalized rates, with a warning that unequal corpus sizes invalidate naïve raw-count comparison.
- Inspect a result row and open the corresponding source passage in BEING; then return to ESSENCE or end the investigation, restoring the original reading position.

BOUNDARIES & CONTRACTS

7. Lifecycle, context, and failure rules

Launch / return contract

Event	Required observable behavior
Open ESSENCE from BEING	Capture cursor/reference; launch Textual asynchronously in a Neovim terminal buffer. No model weights loaded on UI open.
Toggle ESSENCE geometry	Reuse the same active Textual process and logical research context. Recalculate outer and inner layouts on resize.
Focus BEING / ESSENCE	Explicit focus indicator; shortcuts act on focused surface. Escape and quitting are not ambiguous.
Exit ESSENCE	Serialize safe UI/investigation state, terminate only the Textual UI process, close terminal pane, and restore BEING cursor.
Crash / invalid result	Close or recover UI without losing the reader; preserve task logs and provide a visible error. Never silently claim completion.
Long-running model job	If intentionally launched, job policy is explicit: cancel on exit or continue under a separate runner; never depend on the Textual process surviving.

Minimum portable investigation record (proposed)

```
origin: provider=core | file; corpus/edition/generation or file digest
scope: selected UUIDs, codepoint offsets, or file spans
comparison: target scope and optional reference scope
method: task, tokenizer/algorithm/model, exact version
parameters: projection, normalization, seed, thresholds
execution: runner, device, start/end time, job outcome
outputs: artifact digest, warnings, source-linked results
return_to: reader reference, cursor and prior focus/layout
```

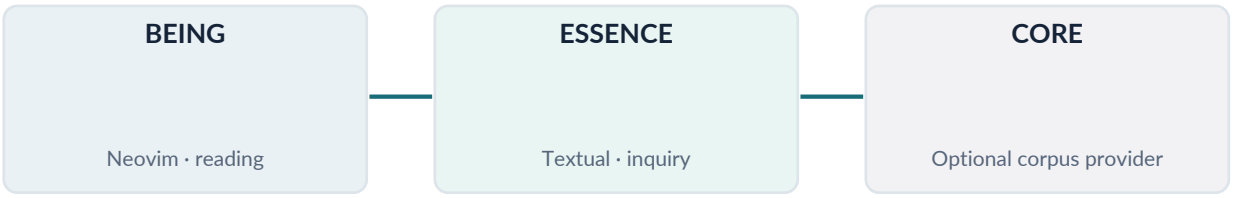
This is a **conceptual schema**, not a proposal to change CORE's version-1 wire protocol. Claude Code should document a compatible boundary and require a separately approved schema/version change before implementation.

One hard architectural limitation

The proposed entry route uses Neovim's embedded terminal and process APIs, which are documented by Neovim. Textual's terminal is then *nested inside* Neovim's terminal buffer. That is plausible, but keyboard escape sequences, terminal modes, alt-key capture, resizing and launch latency need empirical validation on the target terminal. This brief does not substitute for that prototype. [N1]

MODULARITY

8. Independent components without premature packaging



Components may integrate, but ESSENCE must also accept ordinary files.

Target responsibility boundaries. Dependence on CORE is optional for ESSENCE and is a longer-term installation goal for BEING.

ESSENCE owns analytical choices; CORE owns textual evidence

CORE stores and resolves texts, authorship, editions, hierarchy, indexed passages, provenance and lineage. ESSENCE selects analytical targets and reference corpora, performs lightweight transformations where necessary, coordinates existing external libraries/models and stores reproducible results. BEING retains authoring/reading operations. Neither a Textual widget nor a model output should become a canonical source editor.

The key correction from the discussion: ESSENCE is **not itself a general-purpose numerical ML framework**. Its "computational engine" is better described as an **experiment orchestrator**, with task interfaces and adapters to

existing libraries and models (spaCy, scikit-learn, PyTorch, Hugging Face, remote APIs). Simple tokenization/statistical transformations may legitimately execute in ESSENCE, but heavyweight model implementations need not be rewritten.

A practical path to independent installation

- **Now:** keep the single repository but extract pure-text analysis functions whose tests require neither CORE nor a running TUI. Specify interface boundaries and optional providers.
- **Next:** allow ESSENCE CLI/Textual to open ordinary files; CORE integration adds UUID-backed corpus operations as an adapter.
- **Later:** let BEING open ordinary Markdown without forcing a prebuilt CORE corpus. Audit dependencies before proposing separate distributions.
- **Only after stable seams:** split into separately installable `sol-being`, `sol-essence`, and `sol-core` packages if the maintenance and packaging costs justify it.

Do not promise “independently installable now”: the reviewed pyproject.toml still packages SOL as one installable unit. [R1]

LONG-TERM KNOWLEDGE DESIGN

9. The historical investigative encyclopedia

The user’s “synchronic/diachronic” point concerned **the development of machine-learning models themselves**, not historical change in the philosophical corpus. ESSENCE should make it possible to learn why a model was built, what inadequacy its authors identified, how later models responded, and which trade-offs remain. The encyclopedia and the experiment laboratory should be linked without being conflated.

Three separable knowledge objects

Object	Minimum structure	What it is not
Historical entry	Publication year, origin paper, architecture, objective, claimed advance, limitations, evidence, relations to predecessors.	Not proof that newer methods are universally better.
Task/capability record	Tokenization, tagging, embedding, retrieval, classification, generation, evaluation; input/output contracts.	Not a list of architectures or chronological ages.
Runnable adapter/checkpoint	Exact model/checkpoint, implementation package, license, version, expected device, limits, output shape.	Not an encyclopedia claim or an interpretation.

Genealogy is a sourced graph, not a triumphalist timeline

A relationship such as “addresses a weakness of”, “extends”, “changes training objective”, or “trades off accuracy for speed” should carry a citation and confidence/status. For example, an encoder model and a decoder model may be appropriate for different tasks rather than occupying one linear hierarchy. ESSENCE should help users **interrogate** design claims and reproduce comparable tests, not endorse a universal rank order.

What an encyclopedia entry could permit

- Browse BERT, GPT-2 or a later model as a historical object; read the paper, model card, architecture and known limitations.
- Open the declared capabilities and see which tasks are executable with the currently installed optional dependencies.
- Launch a reproducible experiment against a selected passage, a chapter or a reference collection, with model-specific preprocessing clearly visible.
- Compare only **commensurable measures**. Naive Bayes classification and embedding similarity can be displayed side-by-side but their numeric outputs should not be labeled directly comparable without an explicit shared evaluation task.

Do not build this knowledge base as hard-coded Textual screens. Store source-linked structured Markdown or an equivalent lightweight source format; the UI renders it. This is important for stand-alone documentation, version history, contributor review and future migration.

METHOD & CORPUS SCOPE

10. An analysis should not be limited to the highlighted passage

ESSENCE needs two independent choices: **Analysis target** and **Reference corpus**. Valid scopes will eventually include selection, passage, section, chapter, book, author oeuvre, user-defined bibliography, thematic collection, and combined collections. The first milestone need support only scopes the current audited corpus exposes. An important long-term invariant is that a text may belong to multiple collections, so collection membership should not be reduced to a single permanent directory parent.

Scaling constraints to record now

- **Provenance:** editions, translators, source language, attribution and corpus generation must remain available in outputs.
- **Preprocessing:** ordinary word tokens, model-specific subword tokens, sentences and embeddings are distinct representations. Their boundaries and algorithms must be recorded rather than assumed equivalent.
- **Comparability:** show denominators, normalized counts and uncertainty; make language/version/translation mismatches visible.
- **Performance:** large authorship collections require batched/indexed access and caching by stable input/method digest; a TUI should not load an entire corpus into widgets.
- **Legality:** external datasets and copyrighted corpora must remain local or permissioned. A generated artifact should avoid embedding book text unless explicitly authorized.

First experiment should remain elementary

Begin with a deterministic, documented **word tokenizer**, token count, type count, and a frequency table for a selected scope. The point is to test the complete research loop, not establish a definitive theory of linguistic wordhood. Note whether punctuation is excluded, whether case is normalized, and whether Unicode/compound forms are handled. Later add model-specific subword tokenization and morphology as distinct analyses, not as aliases.

DOCUMENTATION CHANGE PLAN

11. What Claude Code should change in the repository

The purpose of the handoff is **documentation reconciliation**, not immediate application development. Canonical design authority remains four Markdown files. Historical documentation stays intact under `docs/legacy/`; `CLAUDE.md` continues to provide operational instructions. [R2,R6]

Canonical file	Required revision / guardrail
docs/ARCHITECTURE.md	State independent module direction; on-demand Textual + persistent Neovim hybrid; ownership of outer vs inner layouts; distinguish immediate prototype from long-term encyclopedia. Preserve current implementation truth.
docs/BEING.md	Add reversible research handoff/return requirements, position and selection preservation, focus and keymap compatibility. Keep reader behavior marked implemented only when tested.
docs/ESSENCE.md	Substantially restructure proposed roadmap: interaction model, encyclopedia genealogy, task registry/adapters, target/reference selector, CLI/headless execution, word tokenizer first. Retain exact current commands and implemented studies.
docs/CORE.md	Clarify corpus/provider contract and long-term composable collection metadata. State current single-edition scope explicitly. Avoid new canonical schema or protocol promises without a versioned proposal.

Required editorial method

- Open the current default branch, pending branches/PRs, coordination task board and implementation files before revising factual claims. Repository documentation may be behind work in progress.
- Label statements consistently as **Implemented**, **Decision**, or **Proposed**. Do not convert the design brief into a false product manual.
- Reconcile contradictory earlier plans and preserve the rationale. Cite legacy pages as history rather than overwriting, deleting or silently copying them into current authority.
- Keep the documentation concise despite the rich vision: one architecture overview, one BEING authority, one ESSENCE authority, one CORE authority. Link to the external research brief as supporting evidence rather than multiplying design authorities.
- Produce a branch/PR and concise handoff including files revised, claims checked against code, old/new semantic differences, unresolved decisions, and checks performed.

Decision ledger: settled versus open

Settled direction	Open decision requiring separate review
Hybrid reader + on-demand Textual	Exact entry shortcut and behavior when initiated from existing `S` sentence study
Two ESSENCE geometries	Default split side, minimum widths, keyboard focus transitions
CORE optional for standalone ESSENCE	Detailed adapter API/versioning and when to split distribution packages
Historical encyclopedia + lab	Source editorial schema, provenance governance, initial model coverage
First real feature = word tokenization	Choice of tokenizer and input scope supported in milestone one
State-preserving return	Exact persistence format; what happens to explicitly started background jobs

SEQUENCING & QUALITY

12. A staged plan that does not build the world at once

Stage	Deliverable	Exit criterion
0. Now: documentation	Reconcile decisions within four canonical docs; preservation and reviews.	Docs checks pass; no application code altered; open decisions identified.
1. Interface shell	Textual mock with Encyclopedia / Methods / Experiment / Results; Neovim entry/exit and split/fullscreen.	Launch on demand, resize without restart, restore reader position; robust keyboard focus.
2. Minimal experiment	Word tokenizer, selectable target scope, inspectable source-linked counts, standalone file input.	Reproducible output from CLI and TUI; no mandatory CORE for plain-text file.
3. Comparison	Reference corpus selector, normalized lexical comparisons, result navigation and export.	Explicit denominators; no false comparability; stale generation refusal.
4. Encyclopedia skeleton	Source-linked entries and navigable model genealogy; no heavyweight runtimes mandated.	History, critique and task descriptions accessible without model downloads.
5. Model adapters	Capability-driven registry, optional runtimes and comparison experiments.	Model versions, input limits, policies and reproducibility documented/tested.

Manual acceptance script for the first future prototype

- Start `sol` in an ordinary terminal. Confirm no Textual process exists until ESSENCE is opened.

- Navigate and select a passage; enter ESSENCE; verify displayed source identity and absence of automatic model imports/downloads.
- Toggle to fullscreen and back repeatedly; verify one active process, unchanged selected scope and reasonable resize behavior at narrow terminal widths.
- Return focus to BEING; perform navigation; reopen ESSENCE with the expected context; exit and restore original source offset.
- Force launch failure, UI exception and job cancellation. BEING must remain usable and the failure must be visible.
- Run the same tokenizer headlessly and from the TUI against identical input; compare input digests, counts, parameters and output.

CRITICAL APPRAISAL

13. Failure modes and mitigations

Risk	Failure mechanism	Control
Nested terminal + focus	Neovim terminal-mode keystrokes collide with Textual and Vim controls.	Explicit focus states, narrow keymap set, real terminal tests, context help.
State drift	UI layout accidentally becomes the owner of selection/analysis state.	Serialize independently of widgets; stable source refs and resume context.
Startup/performance	Importing ML runtimes to draw menus causes latency and GPU allocation.	Lazy imports; capability discovery via metadata; launch benchmark.
Presentation overfitting	A vast encyclopedia gets built as ad hoc panels.	Source-backed entry schema, renderer independent of content, minimal component set.
Historical teleology	Later models wrongly presented as unqualified improvements.	Evidence-linked problem/response/tradeoff relations; compare by task.
Corpus category leakage	Headings, notes, translation metadata mixed into prose embeddings.	Versioned type-aware projections as specified by CORE/ESSENCE.
Illusory independence	Single-package imports prevent stand-alone ESSENCE/BEING.	Core-free unit tests; file input; dependency graph checks.
Documentation inflation	Fresh plan contradicts mature canonical contracts or duplicates old docs.	Four authorities, explicit statuses, preserved legacy and PR review.

The most dangerous outcome is not a technically failing prototype; it is a pleasant-looking TUI that quietly conflates model outputs, source text, research hypotheses and canonical philosophical interpretation. SOL's reference and provenance discipline must survive the interface transition.

AGENT HANDOFF

14. Ready-to-use Claude Code documentation assignment

The paragraph below is suitable as the task description when sharing this PDF. It intentionally requests **documentation changes only**.

```
Inspect SOL main, pending PRs, CLAUDE.md, docs/coordination/,
and the four canonical documents in docs/. Treat this PDF as
research input, not executable implementation authority.

Reconcile today's decisions: persistent Neovim BEING;
on-demand Textual ESSENCE, split/fullscreen presentations,
preserved state, word-tokenizer-first research loop,
CORE-optional standalone ESSENCE, and long-term
model-history encyclopedia + task-oriented model adapters.

Edit docs/ARCHITECTURE.md, BEING.md, ESSENCE.md, CORE.md
only as needed. Clearly mark implemented vs decided vs
proposed. Preserve docs/legacy/ and all existing contracts.
Do NOT implement a Textual UI, ML model or corpus schema.

Work in a separate branch; check links/docs; open a PR.
Provide a concise mapping of decisions to edited sections,
code-verified claims, outstanding questions and test results.
```

If the coordination files already contain assignments for Claude Code or Codex, follow their conflict-avoidance rules and do not overwrite another agent's changes. The report is useful only if its decisions are reconciled against the *current* repository state.

SOURCE NOTES

15. Annotated sources and verification trail

Primary sources are preferred: project repositories, maintainers' documentation and canonical SOL files. The source codes in the report point to the entries below. Links are clickable in most PDF readers. Reviewed 18 October 2026; project details can change.

[R1] SOL packaging manifest — Current single-distribution packaging, optional extras and CLI entry points.

[R2] SOL Architecture — Canonical responsibility split, one-book scope, implementation/proposal status.

[R3] SOL BEING — Neovim reader, Lua modules, focus, state and existing reader keybindings.

[R4] SOL ESSENCE — Lexical/sentence experiments, reproducibility, proposed model research.

[R5] SOL CORE — Source identity, corpus schema, snapshots and provenance.

[R6] SOL legacy policy — Archived-document inventory and canonical documentation authority.

[U1] Herdr terminal multiplexer — Workspaces, panes, detached sessions and process model; verify exact Herdr build in use.

[U2] tmux resize / zoom — `resize-pane -Z` zoom/unzoom interaction.

[U3] Lazygit codebase guide — Contexts, controllers, views, helpers and window arrangement.

[U4] K9s — Resource views, commands, filtering and navigation history.

[U5] Yazi — Selection/preview workflow, Vim-like navigation and async tasks.

[A1] Harlequin — Textual SQL IDE, plugin adapters, companion hsql CLI.

[A2] Posting — Posting 2 = Textual; Posting 3 beta = Go/Terma; forms and command palette.

[A3] Frogmouth — Textual Markdown browser, document navigation/history/bookmarks.

[A4] Toolong — Textual large logs, tabbed files, JSONL and search.

[A5] Dolphie — Database-monitor dashboard and daemon mode without Textual UI.

[A6] Toad — Agent UI, contextual footer/help and documented project status.

[A7] Textualize examples — Textual/Rich, Trogon and other official examples.

[T1] Textual layouts/widgets — Container, Grid, dock and responsive arrangement primitives.

[T2] Textual workers — Concurrency considerations for nonblocking UI work.

[T3] Textual testing — Pilot-driven automated UI interaction tests.

[T4] Textual screens — Logical screens and app navigation distinct from Neovim split geometry.

[N1] Neovim terminal docs — Embedded terminal buffers and asynchronous program launch.

Final recommendation. Claude Code should first incorporate and reconcile these decisions into SOL's four canonical documents. A separate, bounded implementation plan should then prove the two-layout, on-demand Neovim/Textual interaction on actual terminals before adding the encyclopedia, heavyweight models or broad corpus infrastructure.